

ΠΑΝΕΠΙΣΤΗΜΙΟ ΠΕΙΡΑΙΩΣ

UNIVERSITY OF PIRAEUS

Αθηνά Αρμένη, Σωτήρης Χατζηκυριάκου και Βασίλειος Αλέξανδρος
Παναγόπουλος

P20025 P20011 P20150

Εκπαιδευτικό Λογισμικό

Contents

1	Επεξήγηση Κώδικα	2
1.1	Παράδειγμα: Εκπαιδευτικό Λογισμικό για την Εξμάθηση JavaScript	2
1.1.1	Εισαγωγή στην JavaScript	2
1.1.2	Βασικές Έννοιες της JavaScript	2
1.1.3	Προχωρημένες Τεχνικές στη JavaScript	2
1.1.4	Ερωτήσεις / Τεστ Αυτοαξιολόγησης	2
1.1.5	Αποθήκευση Στατιστικών Στοιχείων Προόδου του Μαθητή και Προσαρμοζόμενη Μάθηση	2
2	Αποτελέσματα Προγράμματος	2
3	ΕΠΕΞΗΓΗΣΗ ΚΩΔΙΚΑ	3
3.1	MODELS	3
3.1.1	course model	3
3.1.2	db model	4
3.1.3	level model	9
3.1.4	question model	10
3.2	Service	12
3.2.1	auth gate	12
3.2.2	auth service	14
3.2.3	login or register	21
3.3	ΕΝΧΕΙΡΙΔΙΟ ΧΡΗΣΤΗ	22
3.3.1	Συνδεση στο συστημα	22
3.3.2	Εγγραφή στο συστημα	23
3.3.3	Αρχικο κουίζ	24
3.3.4	Home Page	25
3.3.5	Course Page	26
3.3.6	Quiz Page	27
3.3.7	Sidebar	28

Αναπτύξτε ένα εκπαιδευτικό λογισμικό που στοχεύει στην ενίσχυση της γνώσης και του ενδιαφέροντος για μια γλώσσα προγραμματισμού, όπως Java, Python or JavaScript. Το λογισμικό θα περιλαμβάνει ενότητες διδασκαλίας, πλήθος ασκήσεων αυτοαξιολόγησης με διαφορετικά επίπεδα δυσκολίας, καθώς και δυναμική προσαρμογή του περιεχομένου ανάλογα με την πρόοδο και τις ανάγκες του χρήστη. Ο στόχος είναι να μάθει ο εκπαιδευόμενος τη γλώσσα που θα επιλέξετε και να ενισχυθούν οι δεξιότητές του. Το λογισμικό θα αποθηκεύει επίσης στατιστικά στοιχεία προόδου των χρηστών και θα προσφέρει εξατομικευμένη μάθηση.

1 Επεξήγηση Κώδικα

1.1 Παράδειγμα: Εκπαιδευτικό Λογισμικό για την Εκμάθηση JavaScript

Το εκπαιδευτικό λογισμικό θα μπορούσε να περιλαμβάνει τις παρακάτω ενότητες:

1.1.1 Εισαγωγή στην JavaScript

Στόχος: Εισαγωγή στην ΘααΣκριπτ και τη σημασία της στην ανάπτυξη web.

1.1.2 Βασικές Έννοιες της JavaScript

Στόχος: Διδασκαλία των βασικών εννοιών της ΘααΣκριπτ όπως μεταβλητές, τύποι δεδομένων, συναρτήσεις.
Υλικό: Κείμενο, παραδείγματα κώδικα.

1.1.3 Προχωρημένες Τεχνικές στη JavaScript

Στόχος: Εξερεύνηση πιο σύνθετων θεμάτων όπως οι ασύγχρονες λειτουργίες, τα αντικείμενα και η χειραγώγηση του DOM.

Υλικό: Πρακτικές εφαρμογές και παραδείγματα που δείχνουν πώς οι σύνθετες λειτουργίες ενσωματώνονται σε πραγματικά web projects.

1.1.4 Ερωτήσεις / Τεστ Αυτοαξιολόγησης

Ειδικά Τεστ για Κάθε Ενότητα: Δημιουργία τεστ που καλύπτουν το υλικό κάθε ενότητας, επαναλαμβάνοντας τις βασικές και προχωρημένες έννοιες της JavaScript.

Επαναληπτικά Τεστ: Τεστ που συνδυάζουν ερωτήσεις από προηγούμενες ενότητες για την ενίσχυση της μνήμης και της κατανόησης.

1.1.5 Αποθήκευση Στατιστικών Στοιχείων Προόδου του Μαθητή και Προσαρμοζόμενη Μάθηση

Βάση Δεδομένων: Καταγραφή των απαντήσεων στα τεστ και της επίσκεψης σε κάθε ενότητα.

Διαφορετικά Μονοπάτια Μάθησης: Προσφορά προσαρμοσμένων μαθησιακών διαδρομών βάσει των αποτελεσμάτων και των αναγκών του χρήστη.

Extra: Αναφορές Προόδου: Παρακολούθηση και παρουσίαση της προόδου των χρηστών μέσω ευκρινών γραφημάτων και στατιστικών.

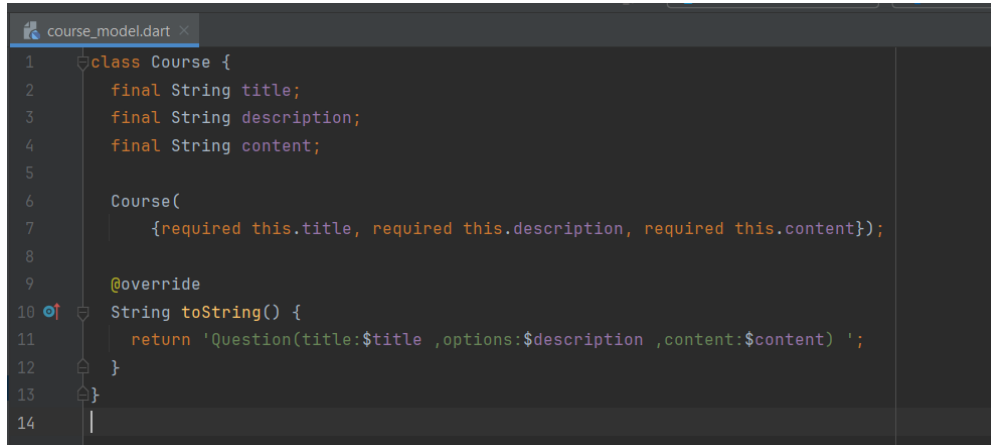
2 Αποτελέσματα Προγράμματος

Στην ενότητα αυτή θα παρουσιαστούν τα αποτελέσματα που προκύπτουν από την υλοποίηση του λογισμικού, συμπεριλαμβανομένων παραδειγμάτων χρήσης, αποτελεσμάτων αυτοαξιολόγησης, και αναφορών προόδου. Επίσης, θα αναλυθούν τα στατιστικά στοιχεία που αποθηκεύονται και τα ανατροφοδοτικά σχόλια των χρηστών.

3 ΕΠΕΞΗΓΗΣΗ ΚΩΔΙΚΑ

3.1 MODELS

3.1.1 course model



```

1  class Course {
2      final String title;
3      final String description;
4      final String content;
5
6      Course(
7          {required this.title, required this.description, required this.content});
8
9      @override
10     String toString() {
11         return 'Question(title:$title ,options:$description ,content:$content) ';
12     }
13 }
14

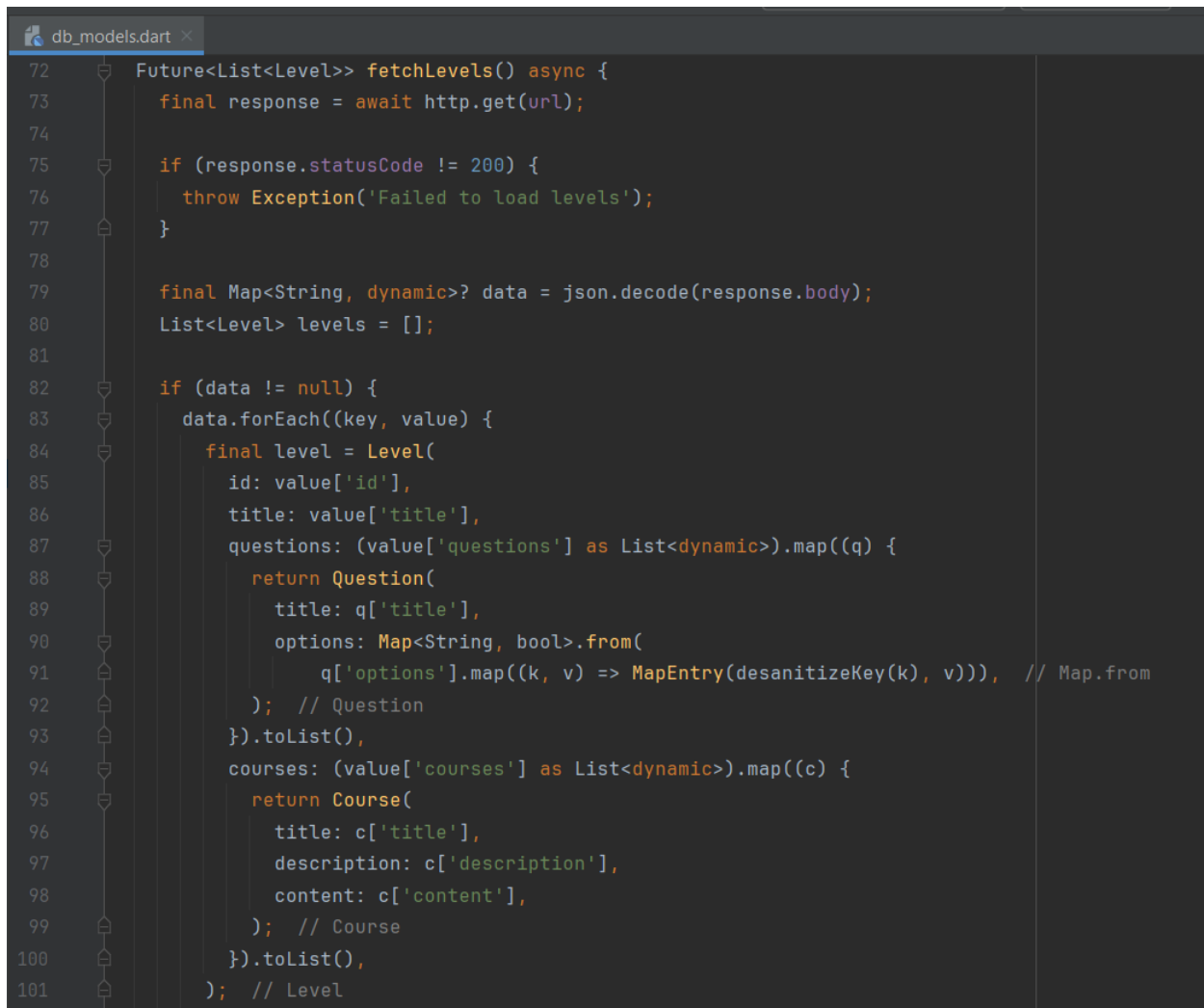
```

- `class Course`: Ορίζει μια νέα κλάση με το όνομα `Course`.
- `final String title`: Δηλώνει μια τελική (αμετάβλητη) μεταβλητή `title` τύπου `String`. Μόλις της αποδοθεί μια τιμή, δεν μπορεί να αλλάξει.
- `final String description`: Δηλώνει μια άλλη τελική μεταβλητή `description` τύπου `String`.
- `final String content`: Δηλώνει μια άλλη τελική μεταβλητή `content` τύπου `String`.
- `@override`: Αυτή η αναφορά δείχνει ότι η μέθοδος παρακάτω επανεγγράφη μια μέθοδο από την υπερκαλή. Σε αυτή την περίπτωση, η `toString` είναι μια μέθοδος από την κλάση `Object`, η οποία είναι η υπερκαλή όλων των κλάσεων στη Dart.
- `String toString()`: Ορίζει τη μέθοδο `toString`, η οποία επιστρέφει μια `String`.
- `return 'Course(title: title, description: description, content: content)'`: Επιστρέφει μια συμβολοσειρά που αντιπροσωπεύει το αντικείμενο `Course`, συμπεριλαμβάνοντας τις τιμές των `title`, `description` and `content`.
- `Course myCourse = Course(...)`: Δημιουργεί ένα νέο αντικείμενο `Course` με τα συγκεκριμένα `title`, `description` and `content`.
- `print(myCourse)`: Εκτυπώνει τη συμβολοσειρά που αντιπροσωπεύει το αντικείμενο `Course` καλώντας τη μέθοδο `toString` έμμεσα.

3.1.2 db model

```
db_models.dart ×
1 import 'dart:convert';
2
3 import 'package:http/http.dart' as http;
4 import 'package:quiz_app/models/course_model.dart';
5 import 'package:quiz_app/models/level_model.dart';
6 import 'package:quiz_app/models/question_models.dart';
7
8 class DBconnect {
9   final url = Uri.parse(
10     'https://quizapp-3b15c-default-rtdb.firebaseio.com/levels.json');
11   final userProgressUrl = Uri.parse(
12     'https://quizapp-3b15c-default-rtdb.firebaseio.com/users/userID/completedCourses.json');
13
14   // Function to sanitize keys by replacing special characters
15   String sanitizeKey(String key) {
16     return key
17       .replaceAll('<', '%3C')
18       .replaceAll('>', '%3E')
19       .replaceAll('.', '%2E')
20       .replaceAll('$', '%24')
21       .replaceAll('[', '%5B')
22       .replaceAll(']', '%5D')
23       .replaceAll('#', '%23')
24       .replaceAll('/', '%2F');
25   }
26
27   // Function to desanitize keys by replacing encoded characters with original characters
28   String desanitizeKey(String key) {
29     return key
30       .replaceAll('%3C', '<')
31       .replaceAll('%3E', '>')
32       .replaceAll('%2E', '.')
33       .replaceAll('%24', '$')
34       .replaceAll('%5B', '[')
35       .replaceAll('%5D', ']')
36       .replaceAll('%23', '#')
37       .replaceAll('%2F', '/');
38   }
39 }
```

```
db_models.dart x
40 Future<void> addLevel(Level level) async {
41   final levelData = {
42     'title': level.title,
43     'questions': level.questions
44       .map((q) => {
45         'title': q.title,
46         'options': q.options.map((key, value) =>
47           MapEntry(sanitizeKey(key), value)), // Sanitize keys
48       })
49     .toList(),
50     'courses': level.courses
51       .map((c) => {
52         'title': c.title,
53         'description': c.description,
54         'content': c.content,
55       })
56     .toList(),
57   };
58
59   final jsonData = json.encode(levelData);
60   print('JSON Data: $jsonData'); // Print the JSON data for debugging
61
62   final response = await http.post(url, body: jsonData);
63
64   if (response.statusCode != 200) {
65     print('Failed to add level: ${response.body}');
66     throw Exception('Failed to add level');
67   } else {
68     print('Successfully added level: ${level.title}');
69   }
70 }
71
```



```
72 Future<List<Level>> fetchLevels() async {
73   final response = await http.get(url);
74
75   if (response.statusCode != 200) {
76     throw Exception('Failed to load levels');
77   }
78
79   final Map<String, dynamic>? data = json.decode(response.body);
80   List<Level> levels = [];
81
82   if (data != null) {
83     data.forEach((key, value) {
84       final level = Level(
85         id: value['id'],
86         title: value['title'],
87         questions: (value['questions'] as List<dynamic>).map((q) {
88           return Question(
89             title: q['title'],
90             options: Map<String, bool>.from(
91               q['options'].map((k, v) => MapEntry(desanitizeKey(k), v))), // Map.from
92             ); // Question
93         }).toList(),
94         courses: (value['courses'] as List<dynamic>).map((c) {
95           return Course(
96             title: c['title'],
97             description: c['description'],
98             content: c['content'],
99           ); // Course
100         }).toList(),
101       ); // Level
```

```

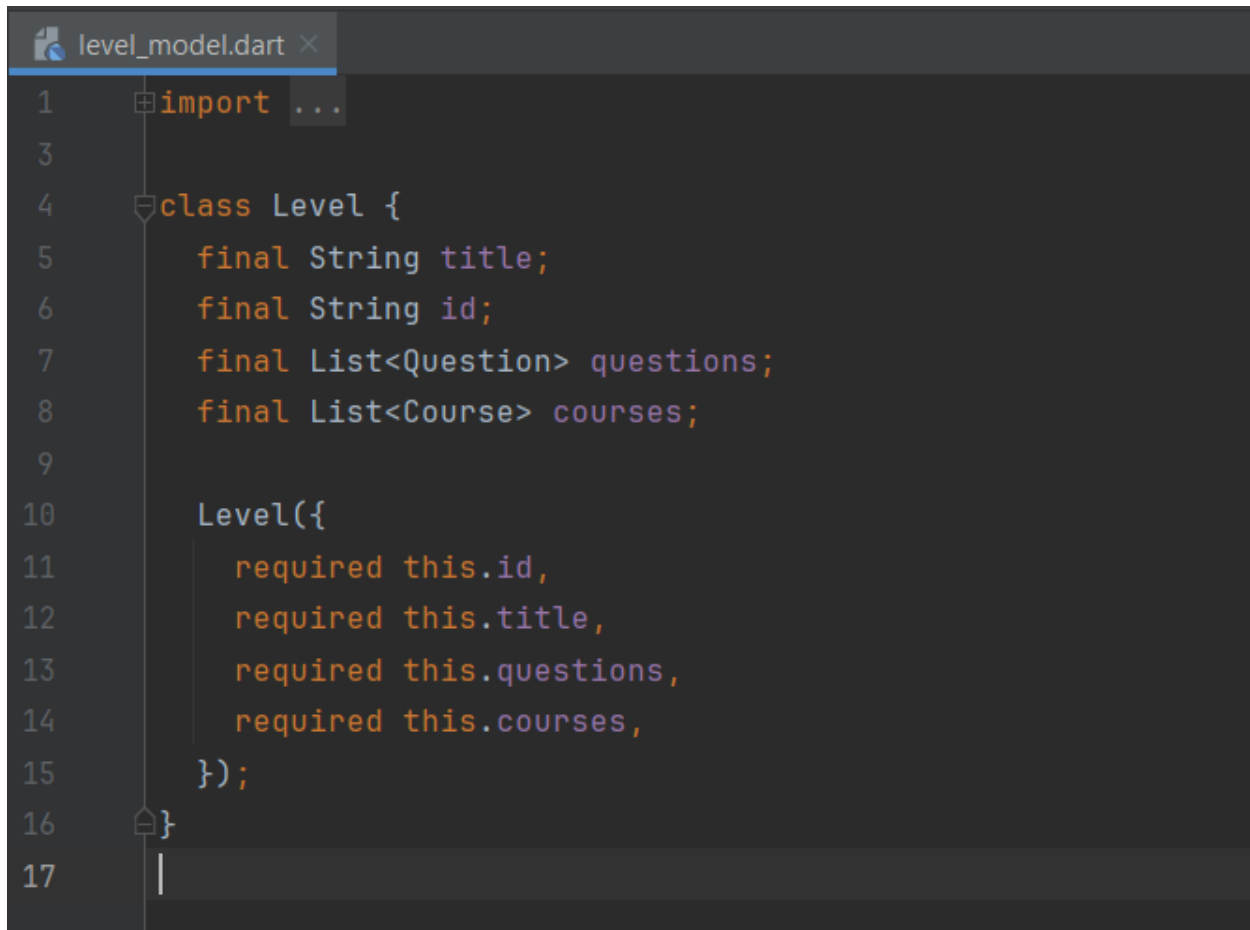
db_models.dart x
110 Future<Level> fetchLevel(String levelKey) async {
111     final sanitizedKey = sanitizeKey(levelKey);
112     final levelUrl = Uri.parse(
113         'https://quizapp-3b15c-default-rtdb.firebaseio.com/levels/$sanitizedKey.json');
114
115     print('Fetching level data from: $levelUrl');
116     final response = await http.get(levelUrl);
117
118     if (response.statusCode != 200) {
119         print('Failed to load level: ${response.statusCode}');
120         throw Exception('Failed to load level');
121     }
122
123     final Map<String, dynamic>? data = json.decode(response.body);
124     print('Fetched level data: $data');
125
126     if (data == null) {
127         throw Exception('Level data is null');
128     }
129
130     final level = Level(
131         id: data['id'],
132         title: data['title'],
133         questions: (data['questions'] as List<dynamic>).map((q) {
134             return Question(
135                 title: q['title'],
136                 options: Map<String, bool>.from(
137                     q['options'].map((k, v) => MapEntry(desanitizeKey(k), v))), // Map.from
138             ); // Question
139         }).toList(),
140         courses: (data['courses'] as List<dynamic>).map((c) {
141             return Course(
142                 title: c['title'],
143                 description: c['description'],
144                 content: c['content'],
145             ); // Course
146         }).toList(),
147     ); // Level
148
149     return level;
150 }
151 }

```

• dart: Παρέχει εργαλεία για την κωδικοποίηση και αποκωδικοποίηση JSON δεδομένων.

- `http`: Πακέτο για την πραγματοποίηση HTTP αιτήσεων.
- `course model.dart`, `level model.dart`, `question models.dart`: Εισαγωγή των μοντέλων που χρησιμοποιούνται στην εφαρμογή.
- `url`: Η βασική διεύθυνση URL για την αποθήκευση των επιπέδων στο Firebase.
- `userProgressUrl`: Η διεύθυνση ΥΡΑ για την αποθήκευση της προόδου του χρήστη στο Firebase.
- `sanitizeKey`: Αυτή η μέθοδος αντικαθιστά ειδικούς χαρακτήρες σε ένα κλειδί με τις αντίστοιχες κωδικοποιημένες μορφές τους για να αποφευχθούν προβλήματα κατά την αποθήκευση στο Firebase.
- `desanitizeKey`: Αυτή η μέθοδος αντιστρέφει τη διαδικασία κωδικοποίησης, μετατρέποντας τις κωδικοποιημένες μορφές πίσω στους αρχικούς χαρακτήρες.
- `addLevel`: Αυτή η μέθοδος προσθέτει ένα νέο επίπεδο στη βάση δεδομένων Firebase.
- Μετατρέπει το αντικείμενο `Level` σε μορφή JSON, χρησιμοποιώντας τη μέθοδο `σανιτιζεΚεψ` για να κωδικοποιήσει τα κλειδιά.
 - Στέλνει ένα HTTP POST αίτημα στη Firebase με τα δεδομένα του επιπέδου.
 - Ελέγχει αν η αίτηση ήταν επιτυχής και εκτυπώνει το αποτέλεσμα.
- `fetchLevels`: Αυτή η μέθοδος ανακτά όλα τα επίπεδα από τη βάση δεδομένων Firebase.
 - Στέλνει ένα HTTP GET αίτημα στη Firebase.
 - Ελέγχει αν η αίτηση ήταν επιτυχής και αποκωδικοποιεί τα δεδομένα JSON σε αντικείμενα `Level`.
- Χρησιμοποιεί τη μέθοδο `δεσανιτιζεΚεψ` για να αποκωδικοποιήσει τα κλειδιά.
- `fetchLevel`: Αυτή η μέθοδος ανακτά ένα συγκεκριμένο επίπεδο από τη βάση δεδομένων Firebase χρησιμοποιώντας το κλειδί του επιπέδου.
 - Χρησιμοποιεί τη μέθοδο `sanitizeKey` για να κωδικοποιήσει το κλειδί.
 - Στέλνει ένα HTTP GET αίτημα στη Firebase.
 - Ελέγχει αν η αίτηση ήταν επιτυχής και αποκωδικοποιεί τα δεδομένα JSON σε ένα αντικείμενο `Level`.
- Χρησιμοποιεί τη μέθοδο `desanitizeKey` για να αποκωδικοποιήσει τα κλειδιά.

3.1.3 level model



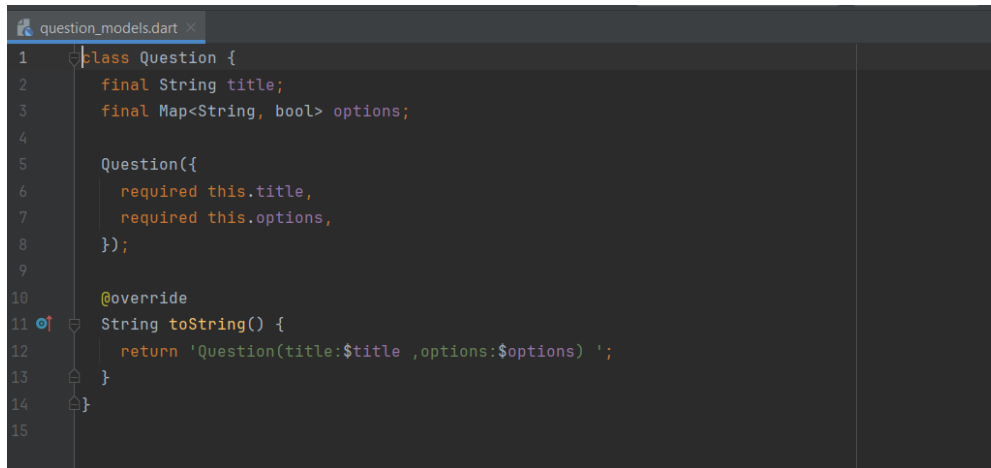
```

1  import ...
3
4  class Level {
5      final String title;
6      final String id;
7      final List<Question> questions;
8      final List<Course> courses;
9
10     Level({
11         required this.id,
12         required this.title,
13         required this.questions,
14         required this.courses,
15     });
16 }
17

```

- `course model.dart`: Εισάγει το μοντέλο `Course`.
- `question models.dart`: Εισάγει το μοντέλο `Question`.
- `class Level`: Ορίζει μια νέα κλάση με το όνομα `Level`.
- Ιδιότητες της Κλάσης
- `final String title`: Δηλώνει μια τελική (αμετάβλητη) μεταβλητή `title` τύπου `String`. Μόλις της αποδοθεί μια τιμή, δεν μπορεί να αλλάξει. Αποτελεί τον τίτλο του επιπέδου.
- `final String id`: Δηλώνει μια άλλη τελική μεταβλητή `id` τύπου `String`. Αποτελεί το μοναδικό αναγνωριστικό του επιπέδου.
- `final List<Question> questions`: Δηλώνει μια τελική μεταβλητή `questions` τύπου `List<Question>`. Αποτελεί μια λίστα από ερωτήσεις που περιλαμβάνονται στο επίπεδο.
- `final List<Course> courses`: Δηλώνει μια τελική μεταβλητή `courses` τύπου `List<Course>`. Αποτελεί μια λίστα από μαθήματα που περιλαμβάνονται στο επίπεδο.
- `Level`: Αυτός είναι ο κατασκευαστής της κλάσης `Level`.
- `required this.id, required this.title, required this.questions, required this.courses`: Αυτές είναι ονομαστικές παράμετροι για τον κατασκευαστή. Η λέξη-κλειδί `required` σημαίνει ότι αυτές οι παράμετροι πρέπει να παρέχονται όταν δημιουργείται ένα αντικείμενο `Level`.
- `List<Question> questions`: Δημιουργεί μια λίστα από αντικείμενα `Question`.
- `List<Course> courses`: Δημιουργεί μια λίστα από αντικείμενα `Course`.
- `Level myLevel = Level(...)`: Δημιουργεί ένα νέο αντικείμενο `Level` με τις συγκεκριμένες `id`, `title`, `questions`, and `courses`.
- `print(...)`: Εκτυπώνει τα στοιχεία του επιπέδου.

3.1.4 question model



```

1 class Question {
2   final String title;
3   final Map<String, bool> options;
4
5   Question({
6     required this.title,
7     required this.options,
8   });
9
10  @override
11  String toString() {
12    return 'Question(title:$title ,options:$options) ';
13  }
14 }
15

```

- **class Question:** Ορίζει μια νέα κλάση με το όνομα Question.
- **final String title:** Δηλώνει μια τελική (αμετάβλητη) μεταβλητή title τύπου String. Μόλις της αποδοθεί μια τιμή, δεν μπορεί να αλλάξει. Αποτελεί τον τίτλο της ερώτησης.
- **final Map<String, bool> options:** Δηλώνει μια τελική μεταβλητή options τύπου Map<String, bool>. Αυτός ο χάρτης (map) περιέχει τις επιλογές της ερώτησης, όπου το κλειδί (τύπου String) είναι το κείμενο της επιλογής και η τιμή (τύπου bool) είναι αν η επιλογή είναι σωστή ή λάθος.
- **Question:** Αυτός είναι ο κατασκευαστής της κλάσης Question.
- **required this.title, required this.options:** Αυτές είναι ονομαστικές παράμετροι για τον κατασκευαστή. Η λέξη-κλειδί required σημαίνει ότι αυτές οι παράμετροι πρέπει να παρέχονται όταν δημιουργείται ένα αντικείμενο Question.
- **@override:** Αυτή η αναφορά δείχνει ότι η μέθοδος παρακάτω επανεγγράφη μια μέθοδο από την υπερκαλή. Σε αυτή την περίπτωση, η toString είναι μια μέθοδος από την κλάση Object, η οποία είναι η υπερκαλή όλων των κλάσεων στη Dart.
- **String toString():** Ορίζει τη μέθοδο toString, η οποία επιστρέφει μια String.
- **return 'Question(title: title ,options: options)':** Επιστρέφει μια συμβολοσειρά που αντιπροσωπεύει το αντικείμενο Question, συμπεριλαμβάνοντας τις τιμές των title and options.
- **Question myQuestion = Question(...):** Δημιουργεί ένα νέο αντικείμενο Question με τον συγκεκριμένο title and options.
- **print(myQuestion):** Εκτυπώνει τη συμβολοσειρά που αντιπροσωπεύει το αντικείμενο Question καλώντας τη μέθοδο toString έμμεσα.

3.2 Service

3.2.1 auth gate

```

1  import ...
6
7  class AuthGate extends StatelessWidget {
8    const AuthGate({super.key});
9
10   @override
11   Widget build(BuildContext context) {
12     return StreamBuilder<User?>(
13       stream: FirebaseAuth.instance.authStateChanges(),
14       builder: (context, snapshot) {
15         if (snapshot.connectionState == ConnectionState.waiting) {
16           return const Center(child: CircularProgressIndicator());
17         }
18
19         if (snapshot.hasData) {
20           User? user = snapshot.data;
21           if (user != null) {
22             return FutureBuilder<String?>(
23               future: AuthService().getNicknameFromFirestore(user.uid),
24               builder: (context, nicknameSnapshot) {
25                 if (nicknameSnapshot.connectionState ==
26                     ConnectionState.waiting) {
27                   // Loading indicator while fetching nickname
28                   return const Center(child: CircularProgressIndicator());
29                 }
30                 if (nicknameSnapshot.hasError) {
31                   // Handle error while fetching nickname
32                   return const Center(child: Text('Error fetching nickname'));
33                 }
34                 if (nicknameSnapshot.hasData && nicknameSnapshot.data != null) {
35                   // Navigate to HomePage with the fetched nickname
36                   return HomePage(nickname: nicknameSnapshot.data!);
37                 } else {
38                   // Handle case where nickname is not found
39                   return const LoginOrRegister();
40                 }
41               },
42             ); // FutureBuilder
43           }
44         }
45         // User is not signed in, navigate to login/register page
46         return const LoginOrRegister();
47       },
48     ); // StreamBuilder
49   }
50 }

```

- `firebase_auth`: Παρέχει τη δυνατότητα αυθεντικοποίησης μέσω Firebase.
- `flutter/material.dart`: Παρέχει τα widgets και το υλικό σχεδιασμού του Flutter.
- `home_page.dart`: Περιέχει την κλάση `HomePage`, την κύρια σελίδα της εφαρμογής μετά την επιτυχή σύνδεση.
- `auth_services.dart`: Περιέχει υπηρεσίες αυθεντικοποίησης, όπως η ανάκτηση του ψευδωνύμου (nickname) από το Firestore.
- `login_or_register.dart`: Περιέχει την κλάση `LoginOrRegister`, την σελίδα σύνδεσης ή εγγραφής.
- `class AuthGate extends StatelessWidget`: Δημιουργεί μια νέα Stateless widget κλάση με όνομα `AuthGate`.
- `const AuthGate(super.key)`: Κατασκευαστής της κλάσης `AuthGate` με έναν προαιρετικό μοναδικό `key`.
- `StreamBuilder User`: Χρησιμοποιείται για να παρακολουθεί τις αλλαγές στην κατάσταση αυθεντικοποίησης του χρήστη μέσω του `FirebaseAuth.instance.authStateChanges()`.
- `stream`: Ορίζει το stream των αλλαγών αυθεντικοποίησης.
- `builder`: Παρέχει ένα callback που κατασκευάζει το widget tree βασισμένο στα δεδομένα του stream.
- `if (snapshot.connectionState == ConnectionState.waiting)`: Ελέγχει αν το `StreamBuilder` περιμένει δεδομένα και εμφανίζει έναν κυκλικό δείκτη φόρτωσης (`CircularProgressIndicator`).
- `if (snapshot.hasData)`: Ελέγχει αν το `snapshot` περιέχει δεδομένα, δηλαδή αν ο χρήστης είναι συνδεδεμένος.
- `User? user = snapshot.data`: Αποθηκεύει τον συνδεδεμένο χρήστη.
- `if (user != null)`: Ελέγχει αν ο χρήστης δεν είναι null.
- `FutureBuilder<String?>`: Χρησιμοποιείται για να παρακολουθεί το `Future` που επιστρέφει το ψευδώνυμο του χρήστη από το Firestore.
- `future`: Καλεί την `AuthServices().getNicknameFromFirestore(user.uid)` για να λάβει το ψευδώνυμο του χρήστη.
- `builder`: Παρέχει ένα callback που κατασκευάζει το widget tree βασισμένο στα δεδομένα του `future`.
- `if (nicknameSnapshot.connectionState == ConnectionState.waiting)`: Ελέγχει αν το `FutureBuilder` περιμένει δεδομένα και εμφανίζει έναν κυκλικό δείκτη φόρτωσης.
- `if (nicknameSnapshot.hasError)`: Ελέγχει αν υπάρχει σφάλμα και εμφανίζει μήνυμα σφάλματος.
- `if (nicknameSnapshot.hasData and nicknameSnapshot.data != null)`: Ελέγχει αν το `future` επέστρεψε δεδομένα και δεν είναι null και εμφανίζει την αρχική σελίδα (`HomePage`) με το ψευδώνυμο του χρήστη.
- `else`: Αν το ψευδώνυμο δεν βρέθηκε, εμφανίζει την σελίδα σύνδεσης/εγγραφής (`LoginOrRegister`).
- `return const LoginOrRegister()`: Εάν ο χρήστης δεν είναι συνδεδεμένος, εμφανίζει την σελίδα σύνδεσης/εγγραφής (`LoginOrRegister`).

3.2.2 auth service

```
auth_services.dart x
1 import 'package:cloud_firestore/cloud_firestore.dart';
2 import 'package:firebase_auth/firebase_auth.dart';
3
4 class AuthService {
5   final FirebaseAuth _firebaseAuth = FirebaseAuth.instance;
6   final FirebaseFirestore _firestore = FirebaseFirestore.instance;
7
8   User? getCurrentUser() {
9     return _firebaseAuth.currentUser;
10  }
11
12  Future<String?> getNicknameFromFirestore(String uid) async {
13    try {
14      DocumentSnapshot<Map<String, dynamic>> userSnapshot =
15        await _firestore.collection('users').doc(uid).get();
16      if (userSnapshot.exists) {
17        return userSnapshot.data()?['nickname'];
18      } else {
19        return null; // User document doesn't exist
20      }
21    } catch (e) {
22      print('Error fetching nickname from Firestore: $e');
23      return null; // Return null in case of any errors
24    }
25  }
```

```
26
27 Future<UserCredential> signInWithEmailPassword(
28     String email, String password) async {
29     try {
30         UserCredential userCredential =
31             await _firebaseAuth.signInWithEmailAndPassword(
32                 email: email,
33                 password: password,
34             );
35         return userCredential;
36     } on FirebaseAuthException catch (e) {
37         throw Exception(e.code);
38     }
39 }
40
41 Future<UserCredential> signUpWithEmailPassword(
42     String email, String password) async {
43     try {
44         UserCredential userCredential =
45             await _firebaseAuth.createUserWithEmailAndPassword(
46                 email: email,
47                 password: password,
48             );
49         return userCredential;
50     } on FirebaseAuthException catch (e) {
51         throw Exception(e.code);
52     }
53 }
```



```
55 Future<void> saveUserToFirestore(String uid, String nickname) async {
56   await _firestore.collection('users').doc(uid).set({
57     'uid': uid,
58     'nickname': nickname,
59     'level': 'beginner',
60     'courses': {
61       '1Beginner': {'0': false, '1': false},
62       '2Intermediate': {'2': false, '3': false},
63       '3Advanced': {'4': false, '5': false},
64       '4Expert': {'6': false, '7': false},
65       '5Master': {'8': false, '9': false},
66     },
67     'quizzes': {
68       '1Beginner': 0,
69       '2Intermediate': 0,
70       '3Advanced': 0,
71       '4Expert': 0,
72       '5Master': 0,
73     },
74   });
75 }
76
77 Future<void> updateUserLevel(String uid, String level) async {
78   try {
79     await _firestore.collection('users').doc(uid).update({'level': level});
80   } catch (e) {
81     print('Error updating user level: $e');
82   }
83 }
```

```
85 Future<void> updateUserProgress(  
86     String uid, String level, int courseId) async {  
87     try {  
88         DocumentSnapshot<Map<String, dynamic>> userDoc =  
89             await _firestore.collection('users').doc(uid).get();  
90  
91         Map<String, dynamic> userData;  
92  
93         if (userDoc.exists) {  
94             userData = userDoc.data()!;  
95         } else {  
96             userData = {  
97                 'uid': uid,  
98                 'nickname': 'defaultNickname',  
99                 'level': 'beginner',  
100                 'courses': {},  
101                 'quizzes': {  
102                     '1Beginner': 0,  
103                     '2Intermediate': 0,  
104                     '3Advanced': 0,  
105                     '4Expert': 0,  
106                     '5Master': 0,  
107                 },  
108             };  
109         }  
110  
111         // Ensure the courses map exists  
112         if (!userData.containsKey('courses')) {  
113             userData['courses'] = {};  
114         }  
115     }
```

```
116 // Update the course completion status
117 if (!userData['courses'].containsKey(level)) {
118     userData['courses'][level] = {};
119 }
120 userData['courses'][level]['$courseId'] = true;
121
122 // Save the updated data back to Firestore
123 await _firestore.collection('users').doc(uid).set(userData);
124 } catch (e) {
125     print('Error updating user progress: $e');
126 }
127 }
128
129 Future<void> signOut() async {
130     return await _firebaseAuth.signOut();
131 }
132
133 Future<void> saveQuizScore(String levelId, int newScore) async {
134     User? user = getCurrentUser();
135     if (user == null) {
136         throw Exception("User not logged in");
137     }
138
139     try {
140         DocumentSnapshot<Map<String, dynamic>> userDoc =
141             await _firestore.collection('users').doc(user.uid).get();
142
143         if (userDoc.exists) {
144             Map<String, dynamic> quizzes = userDoc.data()?['quizzes'] ?? {};
145             int currentScore = quizzes[levelId] ?? 0;
```

```
147         if (newScore > currentScore) {
148             await _firestore.collection('users').doc(user.uid).update({
149                 'quizzes.$levelId': newScore,
150             });
151         }
152     }
153 } catch (e) {
154     print('Error saving quiz score: $e');
155 }
156 }
157
158 Future<String?> getNickname() async {
159     User? user = getCurrentUser();
160     if (user == null) {
161         throw Exception("User not logged in");
162     }
163
164     try {
165         DocumentSnapshot<Map<String, dynamic>> userSnapshot =
166             await _firestore.collection('users').doc(user.uid).get();
167         if (userSnapshot.exists) {
168             return userSnapshot.data()?['nickname'];
169         } else {
170             return null; // User document doesn't exist
171         }
172     } catch (e) {
173         print('Error fetching nickname from Firestore: $e');
174         return null; // Return null in case of any errors
175     }
176 }
177
```

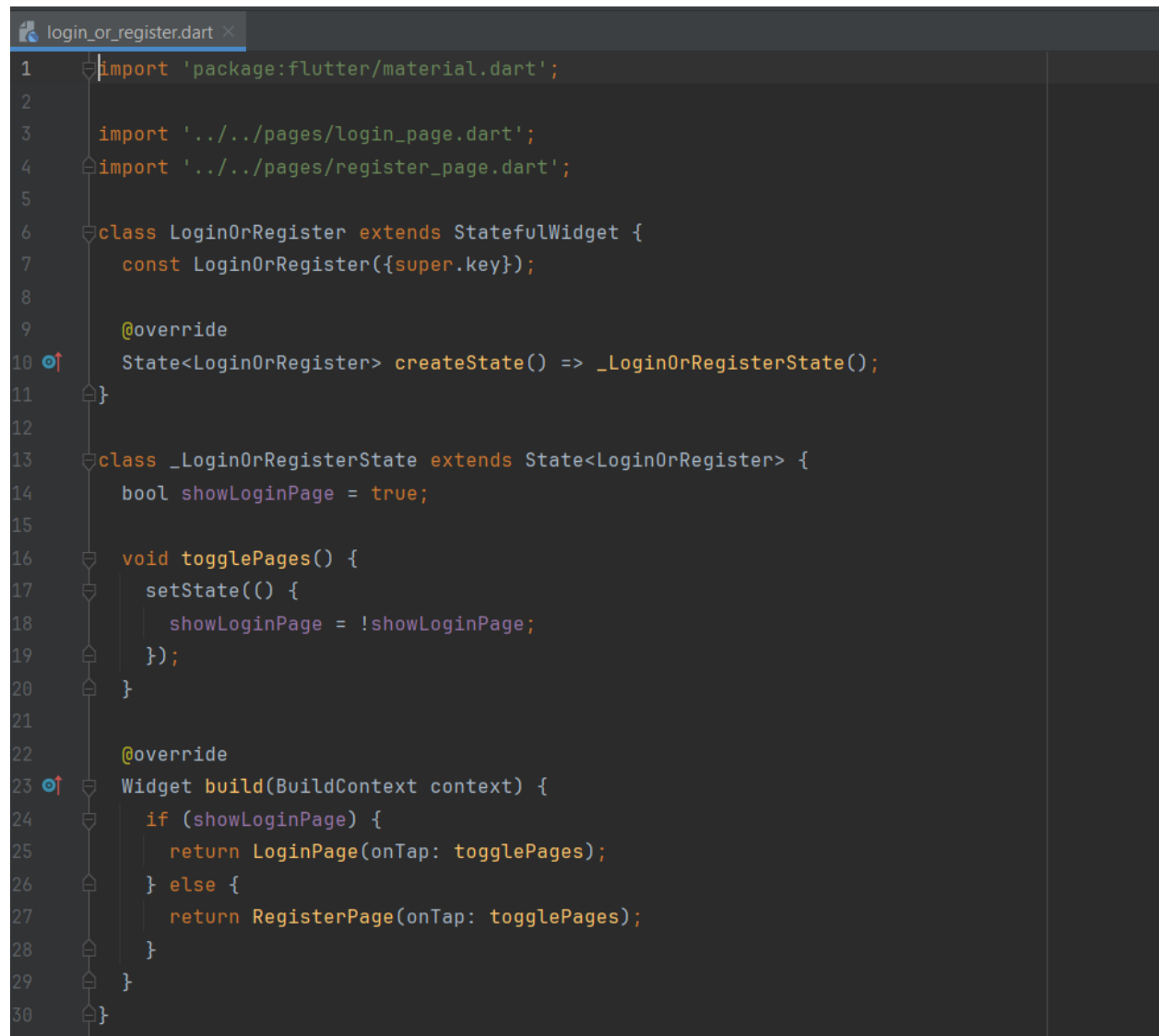
```

178 Future<int> getTotalQuizScore() async {
179     User? user = getCurrentUser();
180     if (user == null) {
181         throw Exception("User not logged in");
182     }
183
184     try {
185         DocumentSnapshot<Map<String, dynamic>> userSnapshot =
186             await _firestore.collection('users').doc(user.uid).get();
187         if (userSnapshot.exists) {
188             Map<String, dynamic> quizzes = userSnapshot.data()?['quizzes'] ?? {};
189             int totalScore = 0;
190             quizzes.forEach((key, value) {
191                 totalScore += value as int;
192             });
193             return totalScore;
194         } else {
195             return 0;
196         }
197     } catch (e) {
198         print('Error fetching total quiz score from Firestore: $e');
199         return 0;
200     }
201 }
202 }

```

- cloud firestore: Παρέχει τη δυνατότητα αλληλεπίδρασης με το Firestore της Firebase.
- firebase auth: Παρέχει τη δυνατότητα αυθεντικοποίησης μέσω Firebase.
- FirebaseAuth firebaseAuth: Μια ιδιωτική μεταβλητή για την αυθεντικοποίησης Firebase.
- FirebaseFirestore firestore: Μια ιδιωτική μεταβλητή για το Firestore.
- getCurrentUser: Επιστρέφει τον τρέχοντα συνδεδεμένο χρήστη (αν υπάρχει).
- getNicknameFromFirestore: Παίρνει το ψευδώνυμο ενός χρήστη από το Firestore με βάση το uid.
- signInWithEmailPassword: Συνδέει έναν χρήστη με email και κωδικό πρόσβασης.
- signUpWithEmailPassword: Δημιουργεί έναν νέο χρήστη με email και κωδικό πρόσβασης.
- saveUserToFirestore: Αποθηκεύει έναν νέο χρήστη στο Firestore με αρχικά δεδομένα.
- updateUserLevel: Ενημερώνει το επίπεδο του χρήστη στο Firestore.
- updateUserProgress: Ενημερώνει την πρόοδο του χρήστη στα μαθήματα στο Firestore.
- signOut: Αποσυνδέει τον χρήστη.
- saveQuizScore: Αποθηκεύει το σκορ ενός quiz στο Firestore.
- getNickname: Παίρνει το ψευδώνυμο του τρέχοντος συνδεδεμένου χρήστη από το Firestore.
- getTotalQuizScore: Παίρνει το συνολικό σκορ των quiz του χρήστη από το Firestore.

3.2.3 login or register



```

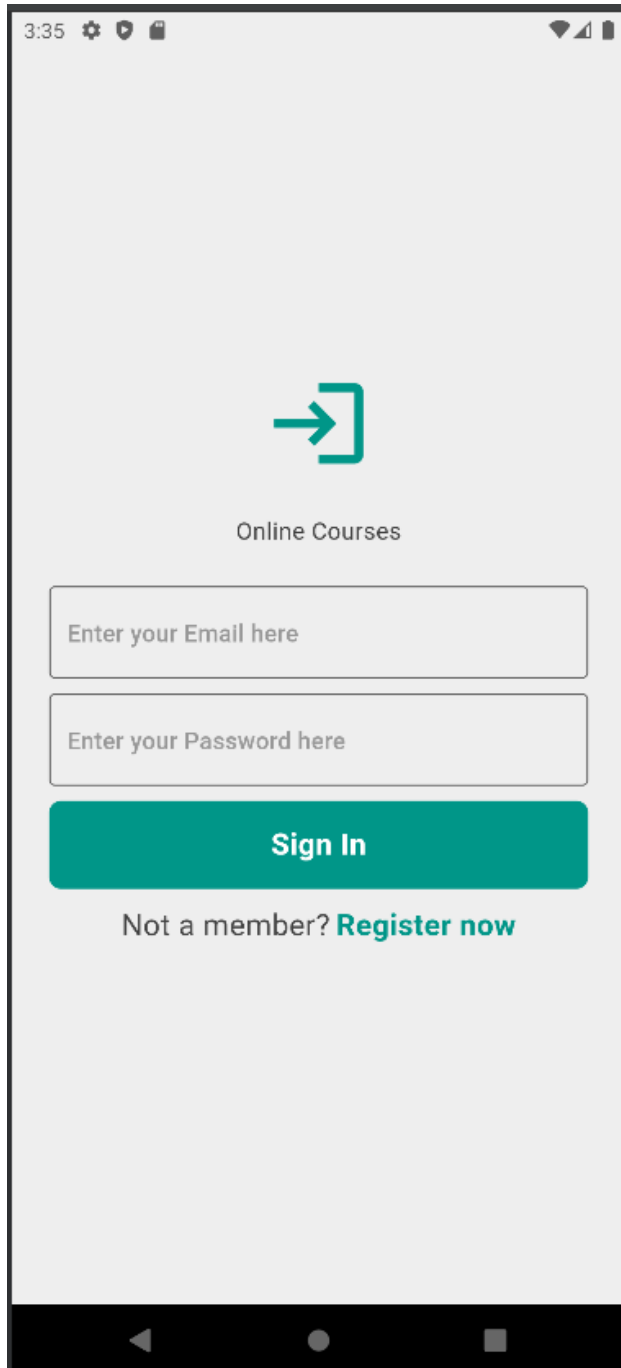
1  import 'package:flutter/material.dart';
2
3  import '../pages/login_page.dart';
4  import '../pages/register_page.dart';
5
6  class LoginOrRegister extends StatefulWidget {
7    const LoginOrRegister({super.key});
8
9    @override
10   State<LoginOrRegister> createState() => _LoginOrRegisterState();
11 }
12
13 class _LoginOrRegisterState extends State<LoginOrRegister> {
14   bool showLoginPage = true;
15
16   void togglePages() {
17     setState(() {
18       showLoginPage = !showLoginPage;
19     });
20   }
21
22   @override
23   Widget build(BuildContext context) {
24     if (showLoginPage) {
25       return LoginPage(onTap: togglePages);
26     } else {
27       return RegisterPage(onTap: togglePages);
28     }
29   }
30 }

```

- flutter/material.dart: Το βασικό πακέτο του Flutter για την κατασκευή του UI.
- ../pages/login page.dart: Εισάγει το αρχείο που περιέχει την κλάση LoginPage.
- ../pages/register page.dart: Εισάγει το αρχείο που περιέχει την κλάση RegisterPage.
- LoginOrRegister: Μια Stateful widget κλάση που διαχειρίζεται την κατάσταση (state) για την εναλλαγή μεταξύ των σελίδων εισόδου και εγγραφής.
- LoginOrRegisterState: Η εσωτερική κατάσταση της κλάσης LoginOrRegister.
- Αυτή η boolean μεταβλητή(showLoginPage) χρησιμοποιείται για να καθορίσει ποια σελίδα θα εμφανιστεί. Αρχικά, είναι true, που σημαίνει ότι η σελίδα εισόδου θα εμφανιστεί.
- Η μέθοδος togglePages εναλλάσσει την τιμή της showLoginPage μεταξύ true and false. Καλώντας την setState ενημερώνεται το UI ώστε να αντανακλά την αλλαγή.
- Η μέθοδος build επιστρέφει είτε το widget LoginPage είτε το widget RegisterPage ανάλογα με την τιμή της showLoginPage.
- LoginPage and RegisterPage: Και τα δύο widgets λαμβάνουν ως παράμετρο τη μέθοδο togglePages μέσω του onTap, ώστε να μπορούν να καλέσουν τη μέθοδο και να εναλλάξουν τις σελίδες.

3.3 ΕΝΧΕΙΡΙΔΙΟ ΧΡΗΣΤΗ

3.3.1 Συνδεση στο συστημα



3:35

Online Courses

Enter your Email here

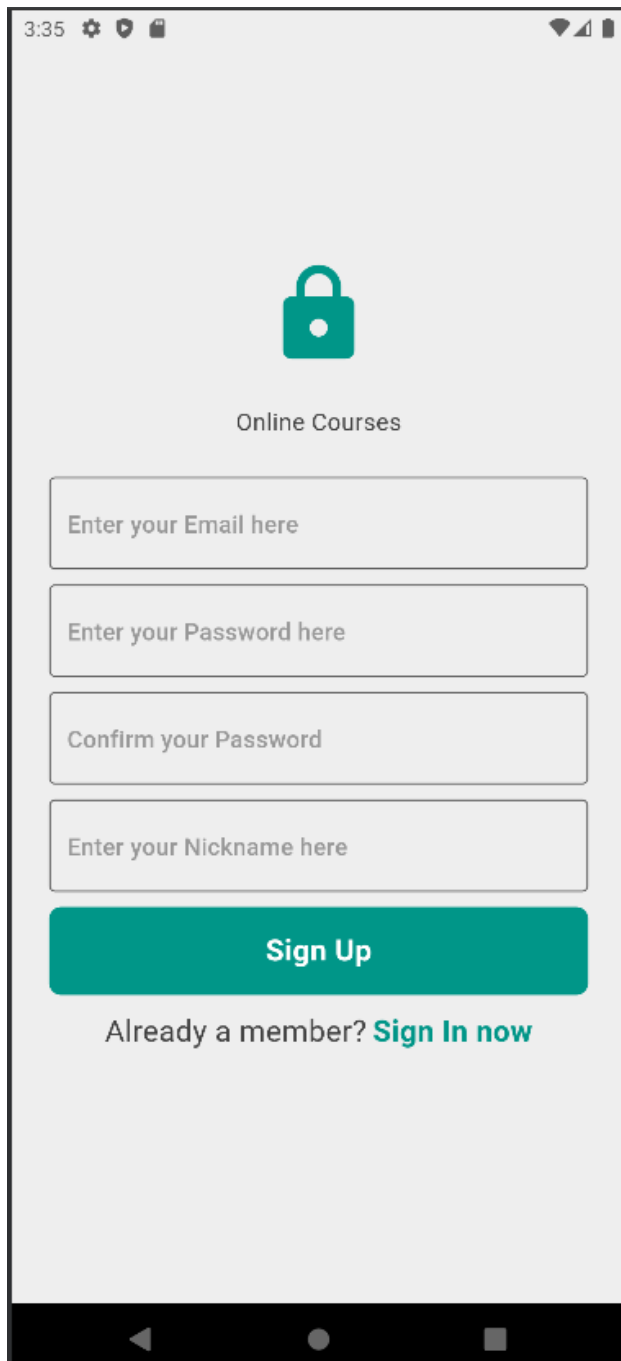
Enter your Password here

Sign In

Not a member? [Register now](#)

Όταν ανοίγει η εφαρμογή, ο χρήστης καλείται να συνδεθεί με τον λογαριασμό του, εισάγοντας το email και τον κωδικό του στα αντίστοιχα πεδία.

3.3.2 Εγγραφή στο σύστημα



3:35

Online Courses

Enter your Email here

Enter your Password here

Confirm your Password

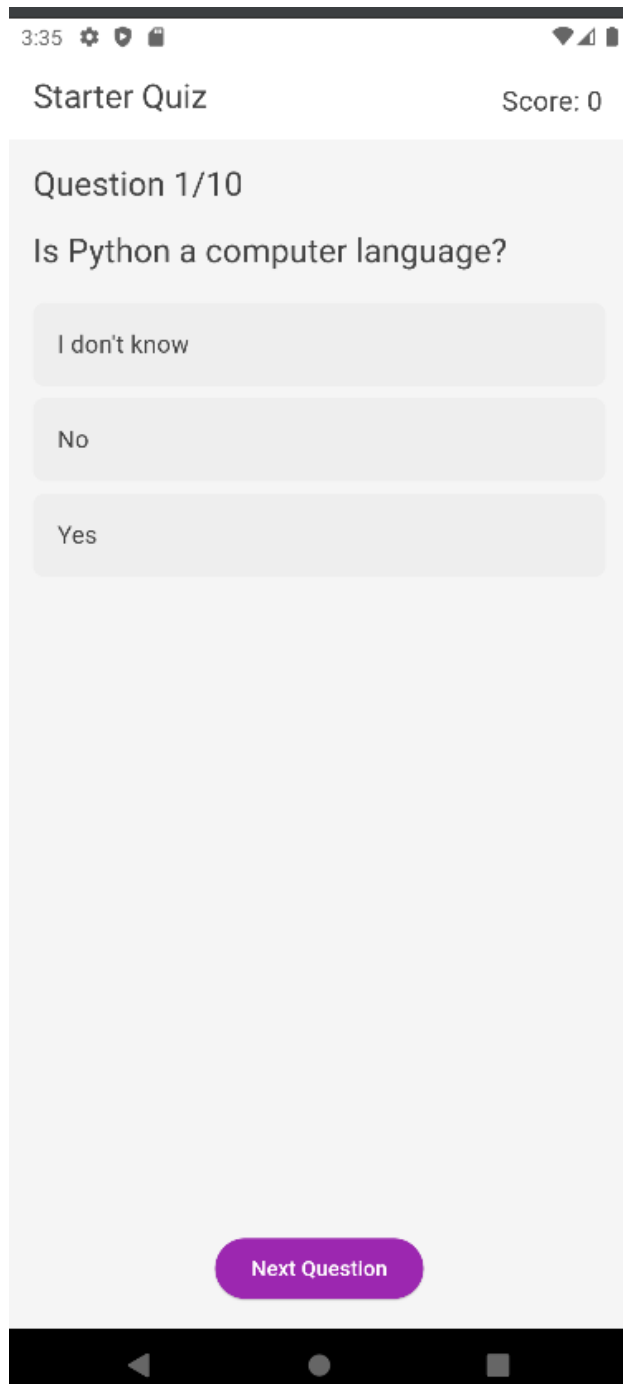
Enter your Nickname here

Sign Up

Already a member? [Sign In now](#)

Στην περίπτωση που ο χρήστης δεν έχει λογαριασμό, μπορεί να πατήσει το κουμπί "Register now" και να μεταφερθεί στην σελίδα της εγγραφής όπου πρέπει να προσθέσει το email του, τον κωδικό του, επαλήθευση του κωδικού και ένα υποκοριστικό της αρεσκείας του. Ύστερα, μπορεί να πατήσει το κουμπί "Sign up" και να ολοκληρωθεί έτσι η εγγραφή του.

3.3.3 Αρχικό κουίζ



3:35 [Settings] [Shield] [App Icon]

Starter Quiz Score: 0

Question 1/10

Is Python a computer language?

I don't know

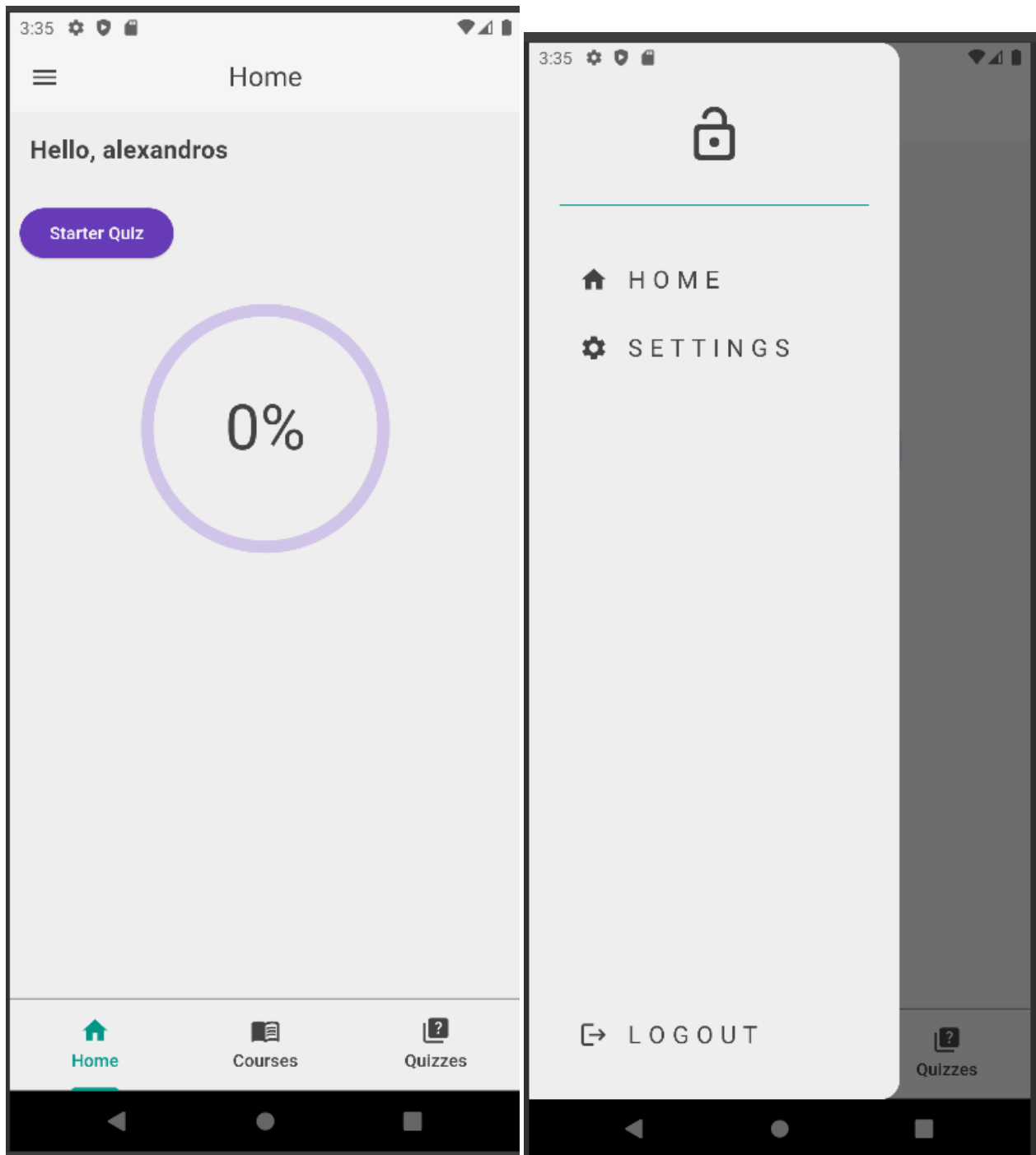
No

Yes

Next Question

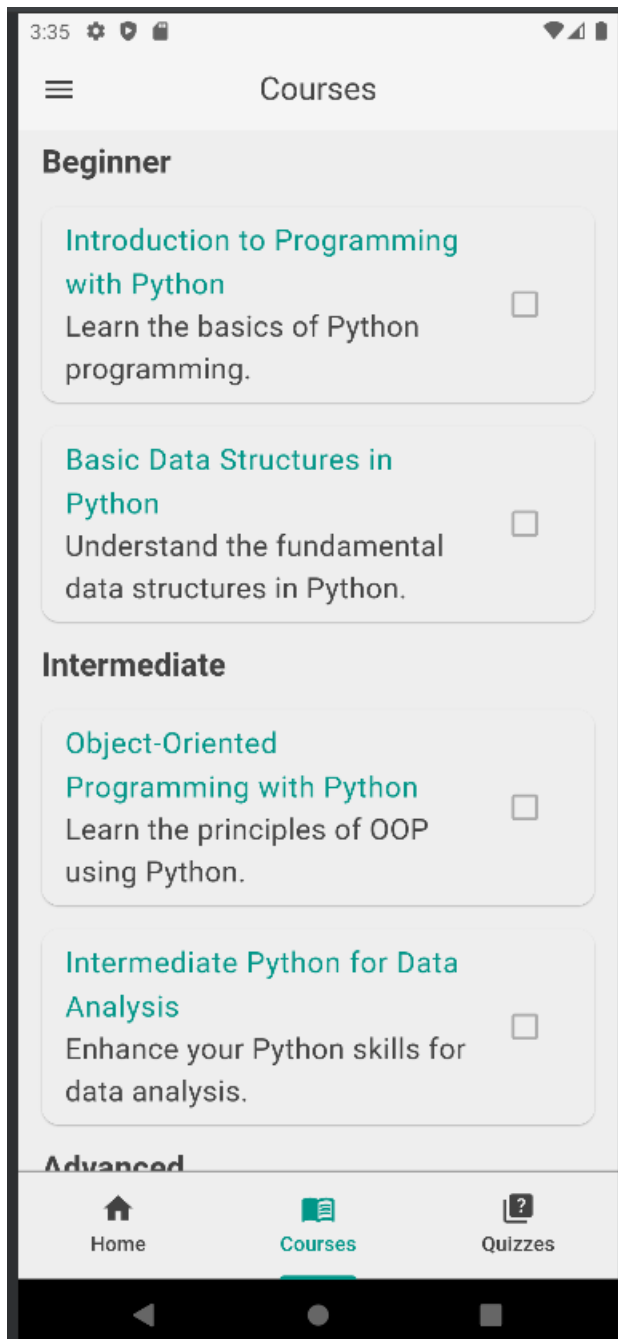
Μετά από την εγγραφή ενός χρήστη, μεταφέρεται σε ένα αρχικό κουίζ για να ελέγξει τις ικανότητές του και να πάρει τον βαθμό που του αντιστοιχεί (Beginner, Intermediate, Advanced, Expert & Master). Αν για κάποιο λόγο δεν κάνει το τεστ τότε λαμβάνει αυτόματα τον βαθμό Beginner. Το τεστ αυτό μπορεί επίσης να το βρει κανείς στο Home page.

3.3.4 Home Page



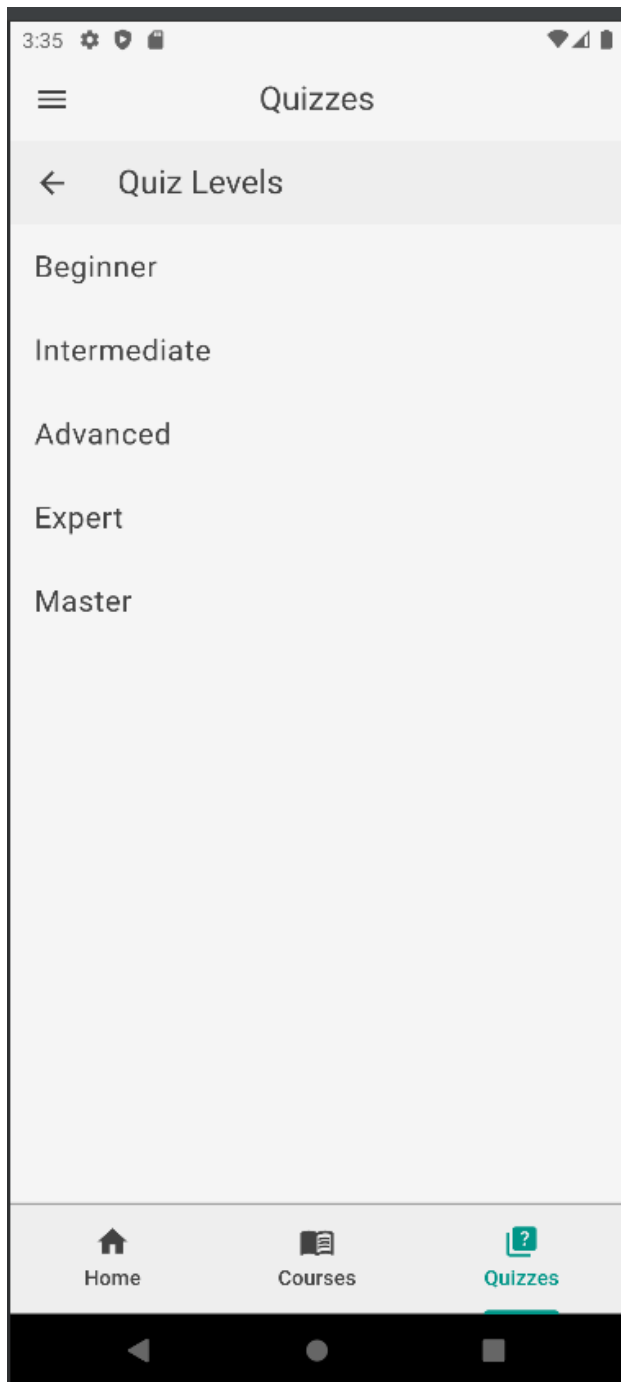
Στην Αρχική σελίδα, μπορεί κανείς να κάνει το αρχικό κουίζ, όπως αναφέρθηκε παραπάνω, και επίσης μπορεί κανείς να δει την πρόοδο του μέσα από έναν κύκλο πρόοδου ο οποίος δείχνει το ποσοστό από το πόσες ερωτήσεις έχουν απαντηθεί σωστά.

3.3.5 Course Page



Στην καρτέλα των μαθημάτων βλέπουμε μια λίστα με όλα τα διαθέσιμα μαθήματα κατηγοριοποιημένα ανά επίπεδο. Κάθε επίπεδο έχει δύο μαθήματα. Επίσης τα μαθήματα μαρκάρονται με ένα τικ ✓ αν τα έχει ολοκληρώσει ο χρήστης. Για να ολοκληρωθεί ένα μάθημα, ο χρήστης πρέπει να πατήσει σε αυτό και τότε θα μεταφερθεί στη σελίδα με το περιεχόμενο του μαθήματος.

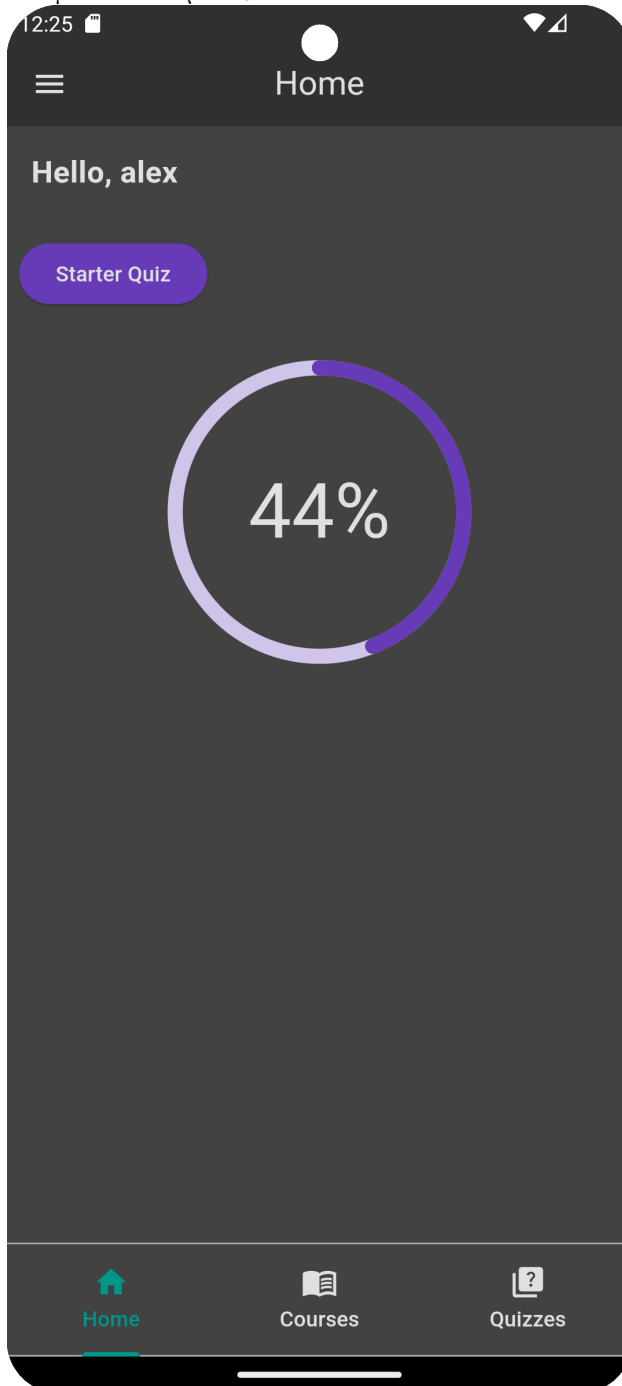
3.3.6 Quiz Page



Στην καρτέλα των κουίζ μπορούμε να δούμε 5 κουίζ με τα ονόματα κάθε επιπέδου. Για να μπορέσουμε να κάνουμε κάποιο κουίζ, πρέπει πρώτα να ολοκληρώσουμε και τα δύο αντίστοιχα μαθήματα για το επίπεδο αυτό. Στα κουίζ βλέπουμε μία ερώτηση και από κάτω έναν αριθμό απαντήσεων. Όταν διαλέξουμε κάποια απάντηση τότε οι λανθασμένες απαντήσεις γίνονται κόκκινες και η σωστή γίνεται πράσινη. Για να πάμε στην επόμενη ερώτηση, πατάμε το κουμπί "Next Question". Στο τέλος του κουίζ, εμφανίζεται ένα κουτί που δείχνει το συνολικό σκορ του κουίζ.

3.3.7 Sidebar

- Home: Σε αυτή την επιλογή απλά γυρνάμε πίσω στο "Home Page"
- Settings Σε αυτή την επιλογή μεταφερόμαστε στα settings και μπορούμε να διαλέξουμε μεταξύ σκούρου και φωτεινού θέματος.



- Logout Σε αυτή την επιλογή απλά κάνουμε log out από την εφαρμογή.